

WatchTower Collector Planning Document

1. Purpose

The collector gathers monitoring data from a user's application and sends it to the WatchTower backend. The system tracks errors, performance issues, user feedback, and deployment information. The architecture follows a collector-style pipeline similar to that of CSE 135: client → collect endpoint → storage → dashboard.

2. Overall Collector Flow

```
User's App
↓
WatchTower Collector Script
↓
POST /collect
↓
Express Backend
↓
MongoDB
↓
Dashboard API
↓
WatchTower Dashboard
```

3. Collector Setup

The collector exposes an `init(config)` function used to initialize configuration for an application.

```
WatchTower.init({
  appId: "my-app",
  apiKey: "abc123",
  release: "1.0.0",
  environment: "production",
  endpoint: "https://watchtower.com/collect"
});
```

The backend uses `appId` and `release` information to associate events with the correct application and deployment version.

4. Main Collector Functions

4.1 `trackError(error)`

Collects JavaScript runtime errors and unhandled promise rejections.

```
WatchTower.trackError(error);
```

Data collected:

```
{
  type: "error",
  appId: "my-app",
  message: error.message,
  stack: error.stack,
  pageUrl: window.location.href,
  userAgent: navigator.userAgent,
  timestamp: new Date().toISOString(),
  release: "1.0.0"
}
```

Browser listeners used:

```
window.addEventListener("error", handler);
window.addEventListener("unhandledrejection", handler);
```

4.2 trackPerformance()

Collects page load and navigation timing information.

```
performance.getEntriesByType("navigation")
```

Data collected:

```
{
  type: "performance",
  appId: "my-app",
  pageUrl: window.location.href,
  loadTimeMs: 1200,
  domContentLoadedMs: 600,
  timestamp: new Date().toISOString(),
  release: "1.0.0"
}
```

4.3 trackFeedback(feedback)

Collects user ratings and feedback messages.

```
WatchTower.trackFeedback({
  rating: 2,
  message: "Page was slow"
});
```

Data collected:

```
{
  type: "feedback",
  appId: "my-app",
  rating: 2,
  message: "Page was slow",
  pageUrl: window.location.href,
  timestamp: new Date().toISOString(),
  release: "1.0.0"
}
```

4.4 trackEvent(eventName, data)

Tracks optional custom application events.

```
WatchTower.trackEvent("checkout_failed", {
  reason: "payment_error"
});
```

Data collected:

```
{
  type: "custom",
  appId: "my-app",
  eventName: "checkout_failed",
  data: {
    reason: "payment_error"
  },
  timestamp: new Date().toISOString(),
  release: "1.0.0"
}
```

5. Sending Data to the Backend

The collector sends monitoring events to the backend using sendBeacon with a fetch

```
fallback.navigator.sendBeacon()
```

Fallback:

```
fetch(endpoint, {
  method: "POST",
  body: JSON.stringify(payload),
  headers: {
    "Content-Type": "application/json"
  },
  keepalive: true
});
```

6. Backend Endpoint

POST /collect

Responsibilities: - Validate payloads - Verify appId/apiKey - Add server timestamp - Store events in MongoDB - Return success response

Example request payload:

```
{
  "appId": "my-app",
  "apiKey": "abc123",
  "type": "error",
  "message": "Button failed",
  "timestamp": "2026-05-07T18:00:00Z",
  "release": "1.0.0"
}
```

7. MongoDB Event Structure

All monitoring data can be stored inside a single events collection.

```
{
  appId: "my-app",
  type: "error",
  timestamp: "2026-05-07T18:00:00Z",
  release: "1.0.0",
  pageUrl: "/dashboard",
  message: "Button failed",
  stack: "...",
}
```

8. Dashboard API Routes

```
GET /api/apps
GET /api/events
GET /api/errors
GET /api/performance
GET /api/feedback
GET /api/overview
```

These endpoints allow the frontend dashboard to fetch monitoring data for a specific application.

9. MVP Scope

The first version should only implement: - `init(config)` - `trackError(error)` - `trackPerformance()` - `trackFeedback(feedback)` - `sendEvent(payload)` - `POST /collect` Additional features should only be added after the core pipeline works reliably.